

Contexto e Solução

Problema

Escritórios de advocacia gastam horas em revisão manual de contratos para identificar cláusulas de risco, extrair informações estruturadas e comparar documentos. O processo é repetitivo, suscetível a erros e consome tempo que poderia ser dedicado a análise estratégica.

Objetivo do MVP

Demonstrar um sistema funcional que acelera a análise contratual com IA, mantendo o profissional jurídico no controle por meio de revisão humana obrigatória.

Público-alvo

Advogados, assessores jurídicos e administradores de escritórios de advocacia.

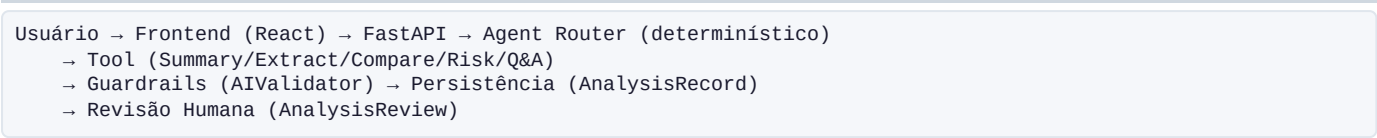
Principais Funcionalidades

- Upload de PDF com extração de texto e chunking
- Chat com agente determinístico (Agent Router) e RAG
- Resumo, extração e comparação via LLM (GPT-4o)
- Análise de riscos heurística (palavras-chave, determinística)
- Guardrails com confidence score, citações e bloqueio de alucinações
- Automação pós-upload com webhook compatível com n8n
- Revisão humana com state machine e histórico append-only
- Métricas de produtividade estimadas
- RBAC com 5 papéis (ADMIN, LAWYER, ASSISTANT, CLIENT, VIEWER)

Stack

Camada	Tecnologia
Backend	FastAPI, Python 3.12, SQLAlchemy, Alembic
Frontend	React, TypeScript, Vite, TailwindCSS, Zustand
Banco	SQLite (PostgreSQL recomendado para produção)
IA	OpenAI GPT-4o via LangChain; text-embedding-3-small
Auth	JWT (HS256) + Argon2

Diagrama de Fluxo



Agent Router — Determinístico

O Agent Router classifica a intenção do usuário por **palavras-chave**, sem chamar LLM. Mapeia para uma de 5 ferramentas. Vantagens: previsibilidade, velocidade e custo zero de tokens.

Intent	Tool	Palavras-chave
SUMMARIZE_DOCUMENT	summarize_document	resumo, resumir, summarize
EXTRACT_INFORMATION	extract_information	extrair, extraia, extract, partes
COMPARE_DOCUMENTS	compare_documents	comparar, compare, diferenças
IDENTIFY_RISKS	contract_risk_analysis	riscos, risco, risk, perigos
QUESTION_ANSWERING	semantic_search	qual, o que, quando, how, what

Engenharia de Prompts

Prompts estruturados em português (pt-BR) para GPT-4o com temperatura 0.3:

- **Resumo:** "Faça um resumo detalhado do seguinte documento legal em português"
- **Extração:** Prompt com schema JSON explícito (parties, dates, values, clauses)
- **Comparação:** Prompt comparativo com estrutura de diferenças
- **Q&A:** Via LangChain Agent Executor com histórico e contexto RAG

Ferramentas e Uso de LLM

Ferramenta	LLM	RAG	AIValidator
summarize_document	Sim (GPT-4o)	Não	Não
extract_information	Sim (GPT-4o)	Não	Não
compare_documents	Sim (GPT-4o)	Não	Não
contract_risk_analysis	Não	Não	Não
semantic_search	Sim (GPT-4o)	Sim	Sim

Pipeline de Documentos

- 1. **Upload:** PDF via POST /documents/upload
- 2. **Extração:** PyPDF2 (sem OCR)
- 3. **Chunking:** Por sentenças (1000 chars, 200 overlap)
- 4. **Embeddings:** OpenAI text-embedding-3-small, armazenados como binário (pickle)
- 5. **Busca:** Similaridade cosseno entre embedding da query e chunks

RAG — Onde é Usado

O RAG é utilizado **apenas** na ferramenta semantic_search (Q&A via chat):

Pergunta → Embedding → Busca cosseno → Top-K chunks → GPT-4o → Resposta + Citações

Análise de Riscos — Heurística (Separada do RAG)

A análise de riscos é **totalmente determinística**, sem LLM e sem RAG:

- **Camada 1:** HeuristicAnalyzer — detecta cláusulas ausentes (confidencialidade, LGPD, rescisão) e padrões problemáticos (multa ilimitada, renovação automática, pagamento indefinido) por palavras-chave
- **Camada 2:** Recuperação textual (não semântica) para gerar citações
- **Camada 3:** Scoring — confidence_score (base 50 + riscos + cobertura) e overall_risk (maior severidade)

Importante: O similarity_score nas citações da análise de riscos é um valor fixo ilustrativo (0.7), não calculado por embeddings.

Guardrails (AIValidator)

Aplicado **apenas em Q&A** (semantic_search):

Componente	Pontos	Critério
Fontes	até 30	Quantidade de chunks recuperados
Similaridade	até 30	Score médio de similaridade
Citações	até 20	Cobertura de citações
Consistência	até 10	Alinhamento resposta-contexto
Qualidade	até 10	Significância do contexto

Bloqueio: Score < 60 → resposta bloqueada, conteúdo não exposto. **Disclaimer:** Sempre presente.

Limitações

- Sem OCR (apenas PDF com texto extraível)
- AIValidator não integrado em resumo, extração e comparação
- Análise de riscos não detecta contradições semânticas

Automação Pós-Upload

Após o upload, `BackgroundTasks` do FastAPI executa:

- 1. **DOCUMENT_PROCESSING** (10%): Verifica documento
- 2. **SUMMARY** (30-50%): Executa `summarize_document` (GPT-4o)
- 3. **RISK_ANALYSIS** (70-85%): Executa `RiskAnalyzer.analyze()` (heurístico)
- 4. **WEBHOOK** (90%): Envia evento `analysis.completed`
- 5. **COMPLETED** (100%): Finaliza `AutomationRun`

Webhook e n8n

- Webhook HTTP POST com payload estruturado (event, document, analysis)
- Idempotency key, retry configurável, timeout
- Workflow n8n de exemplo: `n8n/analysis-completed-workflow.json`
- **Não necessariamente executado em ambiente externo** — workflow de exemplo importável

Revisão Humana

Componente	Descrição
AnalysisRecord	Persiste toda análise (chat, endpoint, automação)
State Machine	GENERATED → PENDING_REVIEW → APPROVED/REJECTED/NEEDS_CHANGES
AnalysisReview	Histórico append-only (nunca modificado ou deletado)
RBAC	ADMIN revisa tudo; LAWYER revisa próprias; ASSISTANT apenas visualiza
Bloqueio	Análises bloqueadas não podem ser aprovadas

Segurança

Aspecto	Implementação
Autenticação	JWT (HS256), access (30min) + refresh (7 dias)
Senhas	Argon2 via passlib
RBAC	5 papéis com verificação por endpoint
Ownership	Usuários acessam apenas próprios documentos/análises
Demo mode	Credenciais demo só visíveis com <code>VITE_DEMO_MODE=true</code>
Secrets	<code>.env</code> em <code>.gitignore</code> ; nenhum secret versionado

Limitação do BackgroundTasks

FastAPI `BackgroundTasks` executa no processo da aplicação. Sem fila externa (Celery/RQ). Em produção com múltiplos workers, tarefas podem ser perdidas em restart. Recomenda-se Celery + Redis para produção.

Testes

166 passed, 0 failed	
Arquivo	Cobertura
test_auth.py	JWT, RBAC, login, refresh, logout
test_api.py	Endpoints, auth enforcement
test_validators.py	Extração, resumo, chat, confiança
test_agent_router.py	Classificação de intenção
test_agent_chat_integration.py	Chat end-to-end com LLM mockado
test_risk_analysis.py	Análise heurística de riscos
test_automation.py	Pipeline, webhook, retry
test_analysis_review.py	Records, reviews, state machine, métricas
test_demo_smoke.py	Fluxo completo de demo, guardrails, RBAC

Validações

- **Migrations:** upgrade/downgrade/re-upgrade validados
- **Frontend:** `tsc && vite build` — PASS
- **Smoke tests:** Fluxo completo (auth → upload → análise → revisão → métricas)
- **Git:** `git diff --check` — sem erros

Métricas Estimadas

Os ganhos de produtividade apresentados são estimativas do MVP e não resultados medidos em produção.

Tempos manuais configuráveis: Resumo (30min), Extração (45min), Comparação (90min), Q&A (15min), Análise de Riscos (120min).

Limitações Conhecidas

1. Análise de riscos é heurística (sem LLM/RAG semântico)
2. `similarity_score` nas citações de risco é fixo (0.7)
3. Sem OCR
4. Sem refresh token auto-refresh
5. Sem testes frontend automatizados
6. SQLite (PostgreSQL recomendado para produção)
7. AIValidator apenas em Q&A (não em resumo/extração/comparação)
8. BackgroundTasks sem fila externa

Próximos Passos

- Integrar AIValidator em todas as operações com LLM
- Adicionar RAG semântico à análise de riscos
- Migrar para PostgreSQL + Celery
- Implementar OCR
- Calibrar métricas com dados reais
- Adicionar testes frontend (Vitest/Playwright)

GitHub

<https://github.com/LeonardoRFragoso/Legal-AI-Copilot>